

RISC-V Debug Module

SystemVerilog RTL Design & Verification

Tool: Vivado 2024.2 | Target: Artix-7 xc7a35ticpg236-1L

1. Project Overview

This project implements a minimal RISC-V Debug Module (DM) compliant with RISC-V External Debug Support Specification v0.13.2. The DM enables external debuggers (GDB/OpenOCD) to halt, inspect, and modify processor state via JTAG without modifying the firmware running on the target processor.

The implementation targets a single-hart RV32 processor with no Program Buffer and no System Bus Access — the minimal configuration required to support halt, resume, GPR read/write, and CSR read/write operations.

1.1 Objectives

- Implement a 5-block DM architecture in SystemVerilog
- Verify each block in isolation using directed testbenches
- Integrate all blocks and verify through GDB-style integration scenarios
- Demonstrate correct operation of halt/resume handshake, abstract command execution, and error handling

1.2 Specification Reference

Specification	RISC-V External Debug Support v0.13.2
Key Sections	Section 3.12 — Register bit layouts
	Section 3.5 — Abstract command encoding
	Section 3.4 — Halt/resume protocol
Tool	Vivado 2024.2 ML Standard (XSIM)
Target Device	Artix-7 xc7a35ticpg236-1L
Language	SystemVerilog (IEEE 1800-2017)

2. System Architecture

The Debug Module consists of five interconnected blocks. The testbench plays the role of both the JTAG TAP controller (driving DMI signals) and the hart stub (responding to register access requests).

2.1 Block Overview

Block	Role	Key Interfaces
dmi_slave	Translates DMI bus transactions into register read/write operations	dmi_op/addr/wdata → reg_wen/addr/wdata
dm_regfile	Smart CSR hub — stores debug state, generates control signals	dmi_slave ↔ FSM ↔ halt_resume_ctrl
abstract_cmd_fsm	Executes abstract commands on the hart	cmd_valid/command → hart_req_*
halt_resume_ctrl	Manages halt/resume handshake with processor	haltreq/resumereq → halt/resume
hart_stub (TB)	Mock processor for simulation	halt/resume/reg_req/reg_ack

2.2 Signal Flow — GPR Read Example

```
GDB writes command=0x00221005 via DMI
→ dmi_slave: reg_wen=1, reg_addr=0x17, reg_wdata=0x00221005
→ dm_regfile: command_r=0x00221005, cmd_valid pulse
→ abstract_cmd_fsm: IDLE→DECODE→EXEC_READ
→ hart_stub: hart_req_valid=1, regno=0x1005 (GPR x5)
→ hart_stub returns: hart_resp_rdata=0xDEADBEEF
→ dm_regfile: data0_r=0xDEADBEEF
GDB reads data0 → receives 0xDEADBEEF
```

3. DM Register Map

Seven registers are implemented per RISC-V Debug Spec Section 3.12. All registers are 32 bits wide and accessed via the DMI interface at 7-bit addresses.

Address	Register	Access	Key Fields
0x04	data0	R/W	Bits[31:0] = data. Mailbox between debugger and FSM.
0x05	data1	R/W	Bits[31:0] = data. Second data register.
0x10	dmcontrol	R/W	Bit[31]=haltreq, Bit[30]=resumereq, Bit[0]=dmactive
0x11	dmstatus	RO	Bit[11:10]=allrunning, Bit[9:8]=allhalted, Bit[3:0]=version(2)

0x12	hartinfo	RO	Returns 32'h0 (minimal implementation)
0x16	abstractcs	R/W	Bit[12]=busy, Bits[10:8]=cmderr (W1C)
0x17	command	W	Bits[31:24]=cmdtype, Bits[22:20]=aarsize, Bit[17]=transfer, Bit[16]=write, Bits[15:0]=regno

4. Part A — DMI Slave & Register File

4.1 DMI Slave (dmi_slave.sv)

The DMI slave translates 3-wire DMI bus transactions into plain register read/write operations for dm_regfile. It is the only block that interfaces with the JTAG TAP controller.

Operation Encoding

```
op=2'b00  NOP    - do nothing
op=2'b01  READ    - read register at addr, return via dmi_rdata
op=2'b10  WRITE   - write wdata to register at addr
op=2'b11  Reserved - treated as NOP
```

Response Encoding

```
resp=2'b00  SUCCESS - operation completed
resp=2'b10  FAILED  - operation failed
resp=2'b11  BUSY    - DM busy, retry
```

4.2 DM Register File (dm_regfile.sv)

The register file is not a simple register array — it is a CSR block with side effects. Key intelligence implemented:

- A3 Reset Behaviour: dmactive=0 blocks all writes except dmcontrol. Ensures no garbage commands execute before DM initialization.
 - cmd_valid Generation: One-cycle pulse generated when command register written while dmactive=1 and !busy. Acts as doorbell to wake FSM.
 - data0 Priority Arbitration: Debugger write takes priority over FSM result. cmd_busy=1 freezes both paths during execution.
 - W1C cmderr: Writing 3'b111 to abstractcs[10:8] clears error. Prevents accidental clearing, protects against FSM race condition.
 - dmstatus Construction: Built combinationally from live halted/running signals. Never stored — always reflects real-time hart state.
 - cmd_wr_while_busy Detection: Pulses when command written while FSM executing. Triggers cmderr=1 (busy error).
-

5. Part B — Abstract Command FSM

The `abstract_cmd_fsm` implements a 5-state Moore FSM using the two-process style (separate combinational next-state and sequential output blocks). Only `cmdtype=0` (AccessRegister) is supported.

5.1 State Encoding

State	Encoding	Action	Exit Condition
ABS_IDLE	3'b000	busy=0, reg_req=0. Wait for cmd_valid.	cmd_valid=1 → DECODE
ABS_DECODE	3'b001	busy=1. Check cmdtype, aarsize, transfer, halted.	valid → EXEC_READ or EXEC_WRITE error → IDLE with cmderr
ABS_EXEC_READ	3'b010	hart_req_valid=1, write=0. Wait for hart response.	hart_resp_valid=1 → DONE
ABS_EXEC_WRITE	3'b011	hart_req_valid=1, write=1, wdata=data0_in.	hart_resp_valid=1 → DONE
ABS_DONE	3'b100	busy=0, cmderr=0. Latch read result into data0_out.	Unconditional → IDLE

5.2 Error Encoding

- `cmderr=1` (busy): New command written while FSM executing. Detected by `dm_regfile cmd_wr_while_busy` signal.
- `cmderr=2` (not supported): `cmdtype != 0`, or `aarsize != 2`, or `transfer != 1`. Detected in DECODE state.
- `cmderr=4` (halt mismatch): `hart_halted=0` when command issued. FSM rejects without accessing hart.

5.3 Command Encoding

```

Read GPR x5:    0x00221005  (cmdtype=0, aarsize=2, transfer=1, write=0, regno=0x1005)
Write GPR x5:   0x00231005  (cmdtype=0, aarsize=2, transfer=1, write=1, regno=0x1005)
Read mtvec:     0x00220305  (cmdtype=0, aarsize=2, transfer=1, write=0, regno=0x0305)
Write mtvec:    0x00230305  (cmdtype=0, aarsize=2, transfer=1, write=1, regno=0x0305)

```

5.4 Key Design Decision — Two-Process FSM

The FSM uses a two-process style: a combinational `always_comb` block computes `next_state`, and a sequential `always_ff` block registers state and computes outputs based on `next_state` (not current state). This ensures outputs are ready one cycle early, matching the timing requirements of the hart handshake protocol.

6. Part C — Halt/Resume Handshake

The `halt_resume_ctrl` implements the glitch-free handshake between the DM and hart. Four requirements are verified:

Req.	Requirement	Verification
C1	<code>haltreq=1</code> asserts halt; <code>allhalted=1</code> only after hart confirms	Waveform: <code>haltreq</code> → <code>halt</code> → <code>halted</code> → <code>allhalted</code> sequence verified
C2	<code>resumereq=1</code> pulses resume; <code>allrunning=1</code> after hart confirms	Waveform: <code>resumereq</code> → resume pulse (1 cycle) → <code>running</code> → <code>allrunning</code>
C3	<code>haltreq+resumereq</code> same cycle: halt wins	Directed test: both asserted simultaneously, <code>halt=1</code> <code>resume=0</code> confirmed
C4	Hart un-halts spontaneously (NMI): DM detects via falling <code>halted</code>	TB drives <code>halted=0</code> without <code>resumereq</code> ; <code>dmstatus</code> reflects immediately

7. Part D — Integration Testbench

The integration testbench (`tb_debug_module`) plays the role of GDB/OpenOCD. It drives DMI transactions directly, simulating what the JTAG TAP would deliver. The hart stub behavior is embedded inline in the testbench.

7.1 Scenario 1 — Full GPR Access Path

Proves end-to-end GPR read/write path through the complete DM.

- Activate DM: write `dmcontrol[0]=1` (`dmactive=1`)
- Verify version: read `dmstatus`, confirm `bits[3:0]=2`
- Halt hart: write `dmcontrol[31]=1` (`haltreq`), wait `allhalted=1`
- Read x5: write `command=0x00221005`, poll `busy=0`, read `data0`
- Write x5=0xCAFEBAFE: write `data0=0xCAFEBAFE`, write `command=0x00231005`
- Read-back x5: verify `data0` returns 0xCAFEBAFE
- Resume: write `dmcontrol[30]=1` (`resumereq`), verify `allrunning=1`

7.2 Scenario 2 — Error Path + Recovery

Proves `cmderr=4` detection and error recovery flow.

- Send command while hart running → verify `cmderr=4`
- Halt hart

- Clear cmderr via W1C: write abstractcs with bits[10:8]=3'b111
- Retry command → verify succeeds with correct data

7.3 Scenario 3 — CSR Access Path

Proves mtvec CSR read/write through regno=0x0305.

- Write mtvec=0xABCD1234: write data0, write command=0x00230305
- Read back mtvec: write command=0x00220305, poll busy=0, read data0
- Verify data0=0xABCD1234

7.4 DMI Transaction Protocol

All DMI transactions are clock-aligned using @(posedge clk) to ensure proper setup/hold timing. The wait_not_busy task polls abstractcs[12] until FSM completes, matching real GDB polling behaviour.

8. Part E — Design Reflection

Q1: Why does the spec mandate dmactive resets everything else?

When dmactive=0 the DM is uninitialized. Registers may contain power-on garbage values. Accepting writes before activation risks executing garbage abstract commands that could corrupt hart state or cause FSM lockup. The dmactive gate ensures the debugger deliberately enables the DM before any operations begin. dmcontrol must remain always writable to avoid the chicken-and-egg problem of needing to write dmactive=1 to activate the DM.

Q2: What breaks if abstractcs.busy isn't set during EXEC?

If busy=0 during execution the debugger reads a stale data0 value and may issue a second command while the first is still executing. This creates a race condition where two commands compete for data0 — one writing the result while the other overwrites the command register. The result is corrupted data0 and potentially two FSM executions overlapping. Setting busy=1 in DECODE (before EXEC begins) serializes all operations correctly.

Q3: How to extend to multi-hart with hartsel?

Add hartsel[25:16] field in dmcontrol. dm_regfile extracts hartsel and routes haltreq, resumereq, and reg_req to the selected hart only. dmstatus uses anyhalted (OR of all halted signals) and allhalted (AND of all halted signals) to reflect per-hart and aggregate status. abs_cmd_fsm needs a hart-select mux to route reg_req to the correct hart stub. hartinfo register returns info about the currently selected hart.

Q4: Why does halt win over resume in the spec?

If resume were to win when both arrive simultaneously, a haltreq could be silently lost — the hart resumes instead of halting, leaving the debugger believing the hart is halted when it is actually running. The debugger would then read registers from a running hart producing corrupted, non-deterministic values. Halt winning guarantees safety: a halt request always succeeds, and the debugger can always retry resume. The reverse is not safe.

Q5: Where is the DMI-to-DM clock crossing and how to cross safely?

The JTAG TAP runs at TCK (1-10 MHz) while the DM runs at system clock (100 MHz+). The crossing occurs at the dmi_req valid signal boundary between the TAP shift register and the DM. Safe crossing requires: (1) a 2-flop synchronizer on the dmi_req valid signal to reduce metastability probability, (2) a gray-code encoded request counter to detect new requests without multi-bit transitions, and (3) a handshake where the DM asserts acknowledge before the TAP captures the response. CDC verification is performed using Spyglass CDC or equivalent tool.

9. Project File Structure

File	Part	Description
dmi_slave.sv	A1	DMI bus decoder, op/addr/data → reg interface
dm_regfile.sv	A2/A3	7-register CSR block with side effects
abstract_cmd_fsm.sv	B	5-state two-process Moore FSM
halt_resume_ctrl.sv	C	Glitch-free halt/resume handshake controller
Integration.sv	D	Top-level debug_module instantiating all blocks
tb_dmi_slave.sv	A	DMI transaction and reset-release waveforms
tb_dm_regfile.sv	A	Register bit layout and A3 reset verification
tb_abs_cmd_fsm.sv	B	3-scenario FSM verification (a)(b)(c)
tb_halt_resume.sv	C	C1-C4 handshake verification
Integration_tb.sv	D	Integration TB with embedded hart stub

10. Key Technical Learnings

10.1 RTL Design

- Two-process FSM style separates next-state logic (combinational) from output logic (sequential) for cleaner synthesis and correct output timing.
- Non-blocking assignments (<=) in always_ff prevent simulation race conditions that blocking assignments (=) would introduce.
- CSR blocks with side effects (cmd_valid pulse generation, W1C semantics, priority arbitration) require careful attention to non-blocking assignment ordering.

- Output registered from next_state (not current state) ensures outputs are ready one cycle before state transitions complete.

10.2 Verification

- Clock-aligned DMI tasks (@(posedge clk) instead of #delay) eliminate setup/hold violations in testbench stimulus.
- wait() statements replace fixed delays for FSM state monitoring — more robust and independent of timing parameters.
- XSIM limitation: for loops inside always_ff reset blocks cause loop variable to become X. Workaround: use separate initial block for memory array initialization.
- Separate unit testbenches for each module isolate bugs before integration, saving debug time significantly.

10.3 Protocol Insights

- DMI is a purpose-built, single-cycle register access protocol — simpler than APB but sufficient for debug bandwidth requirements.
- REQ/ACK handshake between FSM and hart provides clean flow control without requiring knowledge of hart latency.
- W1C (Write-1-to-Clear) pattern for status flags prevents race conditions between hardware setting and software clearing bits.
- The dmactive gate is architecturally critical — it prevents the chicken-and-egg problem while protecting against garbage command execution.

11. Bugs Found and Fixed

Bug	Root Cause	Fix Applied
reg_write=1 in EXEC_READ	IDLE state did not clear reg_write, leaving stale value from previous EXEC_WRITE	Added reg_write<=0 to ABS_IDLE output case
data0_rdata never updates	Checked data0_rdata!=0 to prevent constant overwrite, but breaks when GPR value is zero	Replaced with data0_valid pulse signal from FSM
hart_req_addr=XXXX	regno not latched in DECODE before entering EXEC states, command_reg may be unstable	Added regno_lat register latched in DECODE, used in EXEC states
Output 1 cycle late	Output always_ff used state (current) instead of next_state	Changed case(state) to case(next_state) in output block
GPR init fails in XSIM	for loop inside always_ff reset block causes j variable to become X in XSIM	Moved GPR initialization to separate initial block

dmstatus version wrong	DMI read immediately after write returned stale register value due to 1-cycle latency	Added @(posedge clk) alignment in dmi_read task
------------------------	---	---

12. Conclusion

This project successfully implements a minimal RISC-V Debug Module compliant with Debug Spec v0.13.2. All five architectural blocks are implemented in SystemVerilog and verified through individual unit testbenches and an integration testbench covering three complete GDB-style debug scenarios.

The implementation demonstrates correct operation of: DMI protocol decoding, register file side effects (cmd_valid generation, W1C semantics, dmactive gating), 5-state abstract command FSM execution, halt/resume handshake with all four protocol requirements (C1-C4), and end-to-end GPR/CSR read/write paths.

The project provided hands-on experience with protocol implementation, FSM design patterns, testbench methodology, and simulation debugging — skills directly applicable to memory controller RTL design roles targeting DRAM subsystems and debug infrastructure in modern SoCs.